

Computer Arithmetic and Logic Design

how does a processor implement basic arithmetic operations?

understanding this requires an understanding of **logic circuits**

Combinational logic circuits

(reference: text Sections A.2, A.3)

a combinational logic circuit is a device that implements a **logic function**, i.e. a function that has binary-valued input(s) and output(s)

- 1 can be thought of as corresponding to logical “true”, and 0 to logical “false”

logic functions can be specified using **truth tables**, or with **logic equations** using **Boolean Algebra**

example: a 3-input, 1-output logic function f whose output is 1 if and only if exactly 2 of the inputs are 1

specification with truth table:

A	B	C	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

specification with logic equation:

$$f = \overline{A} \cdot B \cdot C + A \cdot \overline{B} \cdot C + A \cdot B \cdot \overline{C}$$

logic equation uses **operators of Boolean algebra**

$\overline{A}$ 1 if and only if A is 0

$A \cdot B$ 1 if and only if both A and B are 1

$A + B$ 1 if and only if one or both of A and B are 1

laws of Boolean algebra

identity laws:

$$A + 0 = A$$

$$A \cdot 1 = A$$

zero and one laws:

$$A + 1 = 1$$

$$A \cdot 0 = 0$$

inverse laws:

$$A + \overline{A} = 1$$

$$\overline{A} \cdot A = 0$$

commutative & associative laws (both for + and .):

e.g. $A \cdot (B \cdot C) = (A \cdot B) \cdot C$

distributive laws:

$$A \cdot (B + C) = (A \cdot B) + (A \cdot C)$$

$$A + (B \cdot C) = (A + B) \cdot (A + C)$$

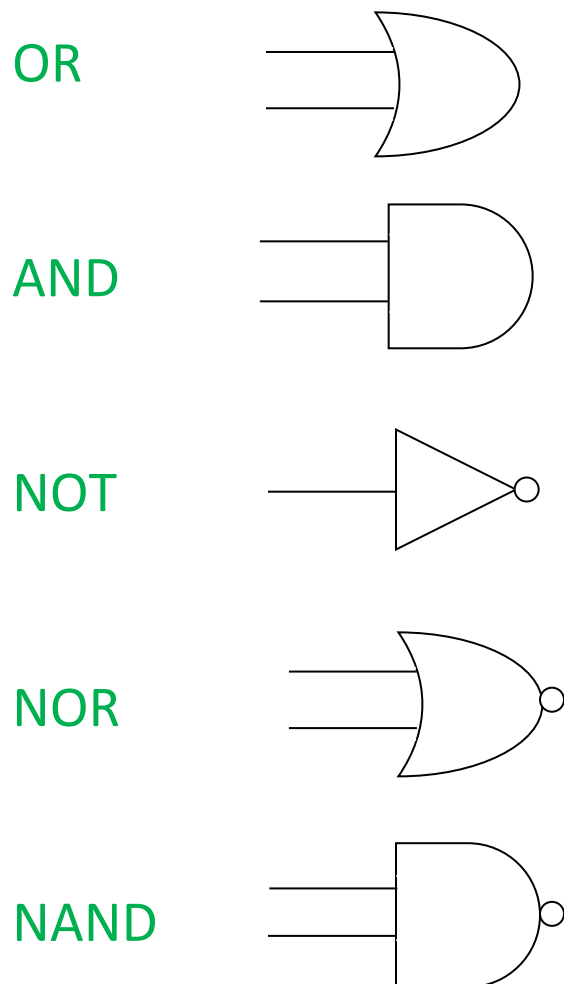
DeMorgan's laws:

$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

$$\overline{A \cdot B} = \overline{A} + \overline{B}$$

implementing logic functions

simple logic functions can be implemented directly with a small number of transistors; these implementations are called **gates** and have standard symbols



more complex logic functions can be implemented with **networks of gates**

one method for designing such a network:

- start with **truth table**
- write **logic equation in sum-of-products form**
- **simplify** using laws of Boolean algebra
- **represent as network** of gates (OR, AND, and NOT)

example: a 3-input, 1-output logic function f called a “**multiplexor**” (or “selector”)

- two **data inputs** A , B ; one “**selector**” input S
- output is identical to A when S is 0
- output is identical to B when S is 1

more generally, a multiplexor with 2^n data inputs would need n selector inputs ($2^n + n$ inputs in total)

multiplexors are very useful for implementing conditional behavior, as we’ll see ...

start with truth table:

A	B	S	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

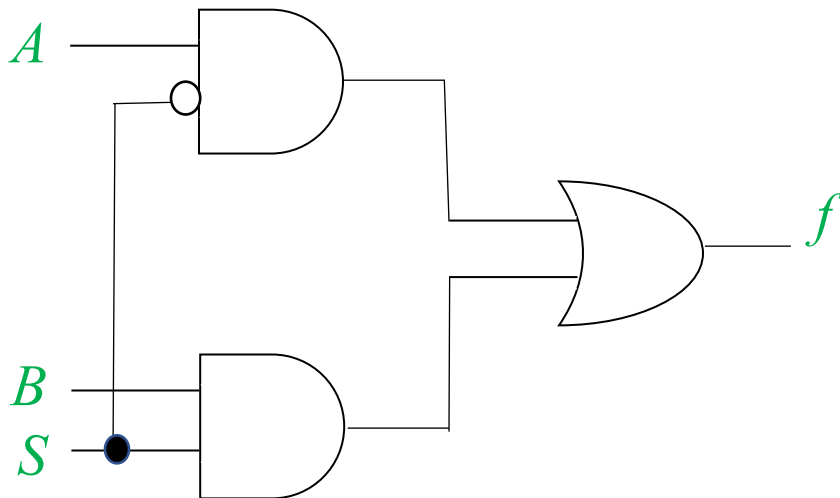
write logic equation in sum of products form, by using **one term for each row** in truth table with **function value of "1"**:

$$f = \bar{A} \cdot B \cdot S + A \cdot \bar{B} \cdot \bar{S} + A \cdot B \cdot \bar{S} + A \cdot B \cdot S$$

simplify:

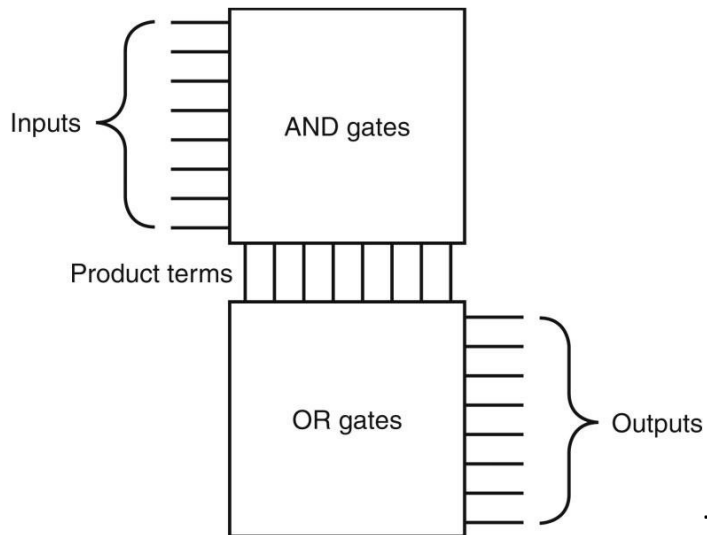
$$\begin{aligned} &= (\bar{A} + A) \cdot B \cdot S + (\bar{B} + B) \cdot A \cdot \bar{S} \\ &= 1 \cdot B \cdot S + 1 \cdot A \cdot \bar{S} \\ &= B \cdot S + A \cdot \bar{S} \end{aligned}$$

represent as network of gates:



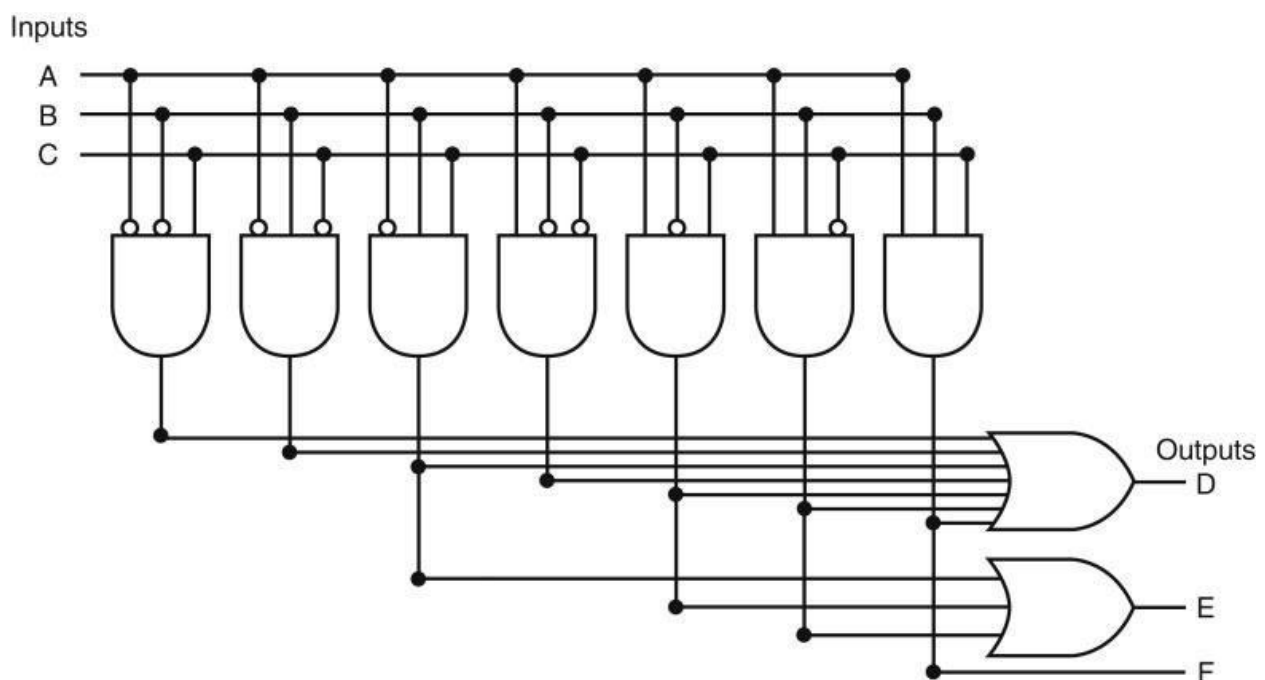
PLAs

rather than using a separate network of gates for each logic function, a structure called a **PLA** (“**programmable logic array**”) can be used



from text Fig. A.3.2

an example (from text Fig. A.3.5):



A simple ALU (“arithmetic logic unit”)

(reference: text Sections A.5, A.6)

initially, will design a simple ALU that implements only the RISC-V “add”, “and”, and “or” instructions

two key ideas in design:

- do **all** operations in parallel, use **multiplexor** to select the desired result
- build 32-bit ALU by using **1-bit ALUs** as building blocks